

# Stopwatch Feature Report

## Keypad- and UART-driven stopwatch for the TM4C123 program

This report describes a stopwatch feature added to the TM4C123GH6PM LaunchPad light program. The stopwatch can be controlled either by three keys on the EduARM4 keypad, wired as genuine hardware interrupts, or by typing commands into a UART terminal.

### 1. Objective

Extend the existing LED blink-rate/colour controller (blinking an RGB LED at a chosen speed and colour, shown on a 4-digit 7-segment display and controllable by on-board switches or UART) with a stopwatch: a timer that can be enabled, started, stopped, paused, and resumed from either a physical keypad or a software console, displaying elapsed minutes and seconds on the same shared display.

### 2. Software Design

The stopwatch is modeled as a four-state machine:

- **DISABLED** — the feature is off; the shared display shows the LED readout instead.
- **IDLE** — enabled, elapsed time at zero, not counting.
- **RUNNING** — actively counting up.
- **PAUSED** — counting frozen, elapsed time retained.

Three actions drive transitions between these states, each reachable from two input paths:

| Action           | Effect and Trigger                                                                                                       |
|------------------|--------------------------------------------------------------------------------------------------------------------------|
| Enable / disable | Toggles between DISABLED and IDLE (enabling always resets elapsed time to zero). Key "1" or UART command SWENABLE.       |
| Start / stop     | From IDLE, starts running. From RUNNING or PAUSED, stops completely and resets to zero. Key "2" or UART command SWSTART. |
| Pause / resume   | From RUNNING, freezes the count. From PAUSED, resumes counting from where it froze. Key "3" or UART command SWPAUSE.     |

Each action's transition logic lives in its own function — `stopwatchEnableToggle()`, `stopwatchStartStopToggle()`, and `stopwatchPauseResumeToggle()` — called identically by both the GPIO Port C interrupt handler and the UART command parser. This ensures the keypad and the console always produce identical behavior, since there is exactly one implementation of each action rather than two independently written ones.

### 3. Timing

A SysTick interrupt fires every 10 milliseconds. It increments elapsed time (in hundredths of a second) whenever the state machine is RUNNING, and it also serves as the timestamp source for debouncing the three keys: any second edge on the same key within 200 ms of the previous one is ignored as switch bounce. The UART command path has no debounce requirement, since each command is only processed once per Enter key press.

### 4. Display Behavior

The 4-digit 7-segment display is shared between two purposes. Ordinarily it shows the LED controller's state: a fixed letter, the current blink rate, the current colour, and a running/paused indicator. The instant the stopwatch

leaves the DISABLED state — by either keypad or UART input — the same four digits switch to showing elapsed time as MM:SS, and they switch back the moment it returns to DISABLED.

## 7. UART Command Reference

| Command  | Effect                                                  |
|----------|---------------------------------------------------------|
| SWENABLE | Enables or disables the stopwatch                       |
| SWSTART  | Starts or stops the stopwatch                           |
| SWPAUSE  | Pauses or resumes the stopwatch                         |
| STATUS   | Reports LED state plus stopwatch state and elapsed time |

Commands connect at 115200 baud, 8 data bits, no parity, 1 stop bit, and are case-sensitive.

## 8. Conclusion

This design adds stopwatch functionality reachable from two independent input paths — genuine interrupt-driven hardware input and a software console interface — while sharing a single implementation of the underlying logic. It shares the display and the SysTick timer with the existing LED logic, introduces no dependency on any external library, and required no changes to the project's startup file.